SAHAYAK | PRESENTATION PLAYBOOK

A role-by-role speaking script for the science exhibition

Three presenters: Shyamantika Pegu, Barakha Doley, Himanxhu V Ghimire
Project: SAHAYAK - A Real-Time Indian Sign Language Recognition System
Prepared for the SAHAYAK project team | October 2026

HOW TO USE THIS PDF

This is a presentation script, not just a project summary. Each member has a clear role, exact speaking lines, what to point at, and a hand-off to the next speaker.
Suggested order: Shyamantika opens and explains the problem; Barakha explains the system and its technology; Himanxhu V Ghimire demonstrates the website, explains limits, and closes. Adjust if your actual work division differs.
Practice the meaning, not every word. Use the live website only after checking that the camera, backend, and model work. Never claim an accuracy score unless measured and documented.

THE STORY IN ONE MINUTE

SAHAYAK aims to make a small set of recognized signs visible as text, with speech support, through a web interface.

The problem: communication between a sign-language user and someone unfamiliar with signs can be difficult. A digital tool may help in some everyday situations, but it is not a replacement for interpreters or full language access.

Current flow: camera -> MediaPipe Hands detects hand landmarks in the browser -> the website converts landmarks into numeric features -> the backend model predicts a supported label -> the page shows text. Speech and translation are separate features that may require internet.

Static two-hand path: 126 values. Continuous-sequence path: 132 values per frame across 10 time bins, for 1320 values.

The current label set should be checked on /api/status before presenting. Project labels have included HELLO, I_LOVE_YOU, NO, THANK_YOU, and YES; do not promise every sign in ISL.

Core honesty rule: say ’prototype’ and ’supported signs’, not ’complete ISL translator’.

No verified model-accuracy figure is supplied in this guide.

SPEAKER 1 | SHYAMANTIKA PEGU

ROLE: OPENING, PROBLEM, PURPOSE AND PROJECT SCOPE

Target time: 1 minute 30 seconds
SAY:
Good morning respected judges, teachers, and everyone present here. We are presenting our project SAHAYAK, a real-time Indian Sign Language recognition system.
The word 'Sahayak' means helper. Our idea is to explore how computer vision and machine learning can support communication between a person using signs and a person who may not understand those signs.
Communication barriers can affect classrooms, public services, and everyday conversations. Sign languages are real languages, but many people around a sign-language user may not know them. Our project explores one small technological aid for this situation.
Our goal is not to replace sign-language interpreters, teach a complete language, or claim that a computer can understand every sign. We are building a prototype that recognizes a limited set of signs and displays the predicted label as text. The website also includes speech and translation features.
We will explain how it works, show the current prototype, and discuss its limitations and future improvements.
DO / SHOW: Display the homepage and point to the project title. Keep this opening slow and confident.
HAND-OFF: Now that we have explained why we chose this problem, Barakha will explain the system and how the recognition pipeline works.

SPEAKER 1 | BACKUP NOTES

KEY TERMS

Computer vision: software that analyses images or video frames.
Landmark: a key point on a hand, such as a fingertip or joint.
Feature: a number that represents useful information from the detected hand.
Classifier: a trained model that chooses among the labels it has learned.
Real-time: the system processes new camera frames while the session is running. Actual speed depends on device, network, and server.

WHY THIS PROJECT MATTERS

It can be used as a learning prototype, a demonstration of accessible technology, and a starting point for a better-tested sign-recognition tool.

DO NOT SAY

- It recognizes every Indian Sign Language sentence.
- It is 98% or 99% accurate unless a valid evaluation proves that.
- It works fully offline. Model inference, translation, or speech may depend on backend or internet.
- It replaces interpreters or guarantees correct communication.

IF ASKED 'WHY SAHAYAK?'

We chose the name because the project is intended to act as a helper. Technology should support accessibility while respecting that sign languages and Deaf communities are much more than a list of gestures.

SPEAKER 2 | BARAKHA DOLEY

ROLE: ARCHITECTURE, AI AND WEBSITE FEATURES

Target time: 2 minutes
SAY:
Thank you, Shyamantika. I will explain how SAHAYAK works.
First, the user opens the website and allows camera access. The browser uses MediaPipe Hands to detect hand landmarks. Instead of sending the whole camera image to the classifier, the current interface extracts numeric hand coordinates and turns them into features.
For the static two-hand model, the feature vector contains 126 values: 21 landmarks for each of two hands, with three coordinates per landmark. These numbers are sent to our Flask backend, where a scikit-learn model saved with Joblib predicts one of its supported labels.
The project also has a continuous-sign pathway. It collects a sequence of frames, represents each frame with 132 values, and resamples the sequence into 10 time bins. That creates a 1320-value input for the sequence model. This is intended for signs where movement over time matters, although results depend on the trained model and quality of the sequence.
The website displays the predicted sign as text. Translation and speech are additional functions; they are not the same thing as recognizing the hand movement. Some of these services need an internet connection.
In short, the system combines browser-based hand tracking, numeric feature extraction, a backend machine-learning model, and a website interface.
DO / SHOW: Use a simple diagram or point through the homepage and Live Studio. Do not start the demo yet.
HAND-OFF: Himanxhu V Ghimire will now demonstrate the live website and explain what the audience should look for.

SPEAKER 2 | TECHNICAL CHEAT SHEET

PIPELINE

1. Camera: captures the user’s hand movement after permission is granted.
2. MediaPipe Hands: estimates hand landmarks in the browser.
3. Feature extraction: converts landmarks into numeric input.
4. Backend API: receives features and runs the model.
5. Classifier: selects a label from the classes it knows.
6. Interface: displays the prediction; speech or translation can be requested separately.

IMPORTANT DIMENSIONS

Static path: 126 features = 2 hands x 21 landmarks x 3 coordinates.

Sequence path: 132 values per frame x 10 time bins = 1320 values.

TECH STACK

Frontend: HTML, CSS, JavaScript and browser-side MediaPipe Hands.

Backend: Python Flask API.

Machine learning: scikit-learn model artifacts saved with Joblib.

Hosting: frontend on Vercel; backend on Render.

WHAT THE MODEL DOES NOT DO

The model does not automatically learn a new sign because a label is typed into a JSON file. New signs require labeled data, training, evaluation, and deployment of a new model artifact.

IF ASKED ’WHY NOT JUST USE IMAGES?’

This version uses hand landmarks as a compact numeric representation. It may reduce dependence on raw image appearance, but can miss facial expression, body posture, and context important in sign language.

SPEAKER 3 | HIMANXHU V GHIMIRE

ROLE: LIVE DEMO, LIMITATIONS, FUTURE WORK AND CLOSING

Target time: 2 minutes plus demo
SAY:
Thank you. I will show the current SAHAYAK prototype.
I am opening Live Studio. The browser requests camera permission. When the hand detector sees a hand, the page can draw landmark annotations. The current model then attempts to classify the extracted features. Please remember that this is a prototype and only its trained labels are supported.
I will make one sign that is listed as supported and hold it clearly in the camera view.
We should look at three things: whether the hand is detected, whether the prediction appears, and whether the output matches the sign I made. A single successful example is a demonstration, not proof of overall accuracy.
Our current limitations include a limited label set, possible confusion between similar signs, sensitivity to lighting and camera position, and the need for broader testing with different signers. Full sign language includes movement, facial expression, body posture, grammar, and context; our prototype does not capture all of that.
Future work includes collecting a larger and more diverse dataset with consent, measuring per-class performance, improving sequence recognition, testing with Deaf and hard-of-hearing users, and adding accessibility and privacy safeguards.
Thank you for listening. We welcome your questions and feedback.
DO / SHOW: Open Studio, check camera permission, make a known supported sign, point out the label and annotation. If it fails, use the backup demo plan.

DEMO CHECKLIST | BEFORE JUDGES ARRIVE

10 MINUTES BEFORE

- Open the homepage and Studio on the actual demo device.
- Confirm internet connection and camera permission.
- Check that the Render backend has woken up and /api/health responds.
- Check /api/status and confirm the model is loaded; note the actual label list.
- Test one or two signs the current model is trained to recognize.
- Check that camera framing shows the hands clearly, with steady lighting and a plain background.

DURING THE DEMO

1. Tell the audience what sign you are about to show.
2. Keep the hand visible and hold the sign naturally; do not move outside the frame.
3. Wait for the prediction. Do not hide a wrong result or pretend a failed result succeeded.
4. If available, show the text output and explain that speech/translation are separate features.

BACKUP PLAN IF THE DEMO FAILS

Say: ’The live result depends on camera detection and the hosted backend. We have a prototype, and this is also a useful example of why reliability testing matters. I can explain the pipeline and show the project interface while we check the connection.’

Then show the homepage or architecture diagram. Do not claim the model worked if it did not.

JUDGE Q&A | PART 1

Q1. What is SAHAYAK?

A: A web-based prototype using hand landmarks and machine learning to recognize a limited set of signs and display predicted labels.

Q2. What problem does it address?

A: It explores a technological aid where someone may not understand a sign-language user's signs. It is not a replacement for interpreters.

Q3. What is MediaPipe?

A: A framework that can detect hand landmarks from camera frames. Here, landmark detection runs in the browser.

Q4. What is a landmark?

A: A key point on a hand, such as a joint or fingertip, represented by coordinates.

Q5. Why use machine learning?

A: The classifier learns patterns in labeled examples and predicts which supported class is most similar to a new feature vector.

Q6. Which language and framework are used on the backend?

A: Python with Flask for the API; the classifier uses scikit-learn and Joblib model files.

Q7. Does it understand all ISL?

A: No. It recognizes only a limited set of trained labels. It is not a complete ISL translator.

Q8. What does real-time mean here?

A: It processes camera frames during a live session. Response time varies with device, network, and backend.

JUDGE Q&A | PART 2

Q9. Why are there 126 features?

A: Two hands x 21 landmarks per hand x 3 coordinates per landmark equals 126 values.
Q10. Why is there a sequence model?
A: Some signs depend on movement over time, so a sequence can represent more than a single hand pose. Success still depends on data and training.
Q11. Does the project work offline?
A: Not fully. Some browser-side functions can run after loading, but backend prediction, translation, or speech may require connectivity.
Q12. How do you know its accuracy?
A: Report accuracy only after a documented test on held-out examples, ideally separated by signer or recording session. We do not claim an unverified percentage.
Q13. How will you add a new sign?
A: Define the label, collect diverse labeled samples, train a candidate model, evaluate it against old and new signs, update label metadata and model artifact, then deploy and test.
Q14. Is typing a new label enough?
A: No. The model must be trained on examples of that sign. A label file alone does not teach the model.
Q15. What are the main limitations?
A: Limited signs, possible errors, sensitivity to recording conditions, network dependence for some features, and missing context such as facial expression and grammar.
Q16. How do you protect users?
A: Ask for camera permission, explain data processing, avoid retaining video unnecessarily, and get informed consent for training data.

JUDGE Q&A | PART 3

Q17. Why not translate whole sentences?

A: This prototype predicts limited labels. Sentence-level sign-language translation needs richer data and models for grammar, context, movement and other visual cues.

Q18. Can it replace a human interpreter?

A: No. It is a limited assistive prototype and should not be relied on for high-stakes conversations.

Q19. Difference between recognition and translation?

A: Recognition predicts a sign label. Translation converts text or a recognized label into another language; that does not mean the system understood a full signed sentence.

Q20. What will you improve next?

A: Expand and balance the dataset, test across signers and lighting, publish honest metrics, improve sequence recognition, and conduct usability testing with the intended community.

Q21. What if the prediction is wrong?

A: The user should be able to repeat or correct the message. Predictions should not be treated as certain, especially in important conversations.

Q22. What makes this scientific?

A: It has a defined problem, technical method, testable outputs, limitations, and a plan to evaluate performance rather than relying on a visual demo alone.

Q23. Why is diverse data important?

A: Hands, signing styles, speed, camera angles, and backgrounds vary. A model trained on a few people may fail for others.

Q24. Main conclusion?

A: Computer vision can support a limited sign-recognition prototype, but meaningful accessibility requires accuracy testing, user involvement, privacy, and respect for sign-language complexity.

FINAL REHEARSAL PLAN

RUN-THROUGH 1: SCRIPT

Each member reads their part aloud. Keep transitions smooth and avoid repeating the same explanation.

RUN-THROUGH 2: WITHOUT READING

Use only the role headings and technical cheat sheet. Explain ideas in your own words.

RUN-THROUGH 3: DEMO

Practice camera permission, one supported sign, waiting for output, and the backup plan if the server is slow.

TEAM RULES

- Do not interrupt another member; use the hand-off line.
- If a judge asks a question, the member who knows can respond, but keep the answer short and accurate.
- If you do not know, say: 'We have not tested that yet, so we do not want to guess. It is part of our future work.'
- Be clear about each member's real contribution. This role split is a suggested presentation division, not a claim about who coded each feature.

FINAL LINE FOR ALL THREE

Together: 'SAHAYAK is a work in progress, and our aim is to keep improving it through careful testing and feedback. Thank you.'

PROJECT LINKS

Live website: https://sahayak-eight-lac.vercel.app
Live Studio: https://sahayak-eight-lac.vercel.app/studio.html
Source code: https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL
Backend health: https://sahayak-backend-krh1.onrender.com/api/health
Backend status: https://sahayak-backend-krh1.onrender.com/api/status